

Cache Address Encryption for Secure RISC-V Processor

Advanced Computer Architecture

Rupak Antani
2022102045
rupak.antani@students.iiit.ac.in

Abhinav S
2022102037
abhinav.s@students.iiit.ac.in

Abstract—Modern computer systems face many security vulnerabilities, including TLB attacks, cache side-channel attacks and speculative attacks. Among these, cache side-channel attacks have become more important recently because they exploit how the cache works to access sensitive data without directly accessing the victim's system. These attacks are easier to execute as they take advantage of hardware features.

Cache side-channel attacks exploit unintended information leaks from the cache, allowing attackers to deduce sensitive data by observing cache behavior. Conflict-based cache side-channel attacks exploit cache conflicts caused when multiple processes compete for the same cache resources. Attackers use techniques like Prime+Probe and Evict+Reload to observe cache behavior and infer accessed memory addresses.

To defend against such attacks, one approach is to encrypt the mapping of physical addresses to cache addresses. This is illustrated using a simple XOR-based encryption algorithm. It can be used to make the mapping unpredictable, but it has some weaknesses, such as allowing advanced attacks. To overcome these problems, a timing noise extension mechanism is introduced, which randomly adds noise to cache operations to confuse attackers.

This project focuses on implementing an encryption mechanism using AES for secure address mapping. The design is tested on an FPGA and integrated with a Linux-based system to evaluate its security and practical use.

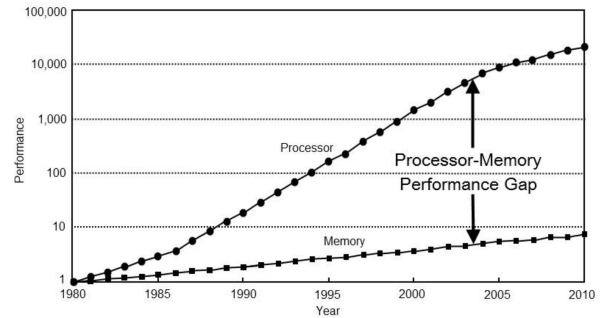
I. INTRODUCTION

A. Memory Wall

Over time, a significant performance gap has emerged between processors and memory units, with processor speeds outpacing memory improvements as illustrated in Fig 1. This discrepancy creates a bottleneck in overall computer performance.

B. Cache Hierarchy

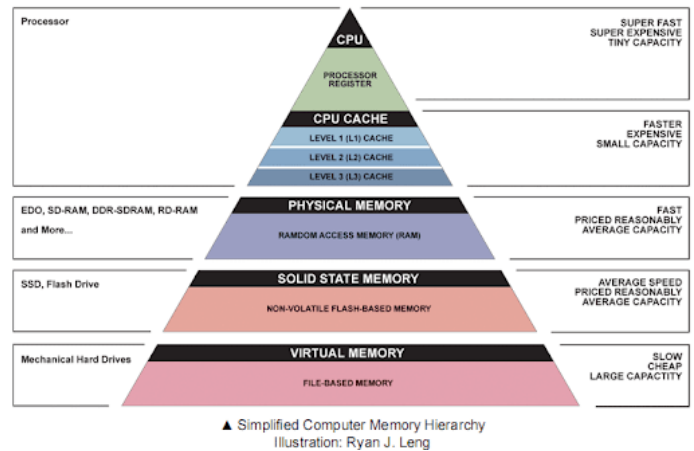
To address the growing performance gap between processors and memory, cache memories were introduced as shown in Fig 2. These provide faster data access compared to main memory while being smaller and more cost-effective. Caches are arranged hierarchically as shown in Fig 3: L1 caches, located closest to the core, are the fastest but smallest, typically used for immediate data and instructions. L2 caches are larger and slightly slower, offering additional storage. The last-level cache (L3 or LLC) is the largest and slowest in the hierarchy, shared across cores, and serves as the final buffer before



Computer Architecture: A Quantitative Approach by John L. Hennessy, David A. Patterson, Andrea C. Arpaci-Dusseau

Fig. 1: Memory Wall

accessing main memory. This layered design balances speed, capacity, and cost to optimize performance.



▲ Simplified Computer Memory Hierarchy
Illustration: Ryan J. Leng

Fig. 2: Computer Memory Hierarchy

C. Locality Principle

When a program accesses a memory location, it is likely to access it again soon, a pattern known as temporal locality. Similarly, accessing one memory location often leads to accessing nearby locations, known as spatial locality. To leverage these patterns, caches store frequently accessed data and transfer

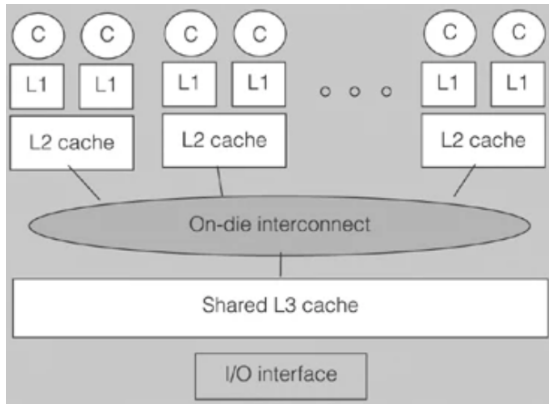


Fig. 3: Memory hierarchy in Multi-core processors

entire memory blocks to include nearby data, speeding up future accesses.

D. Cache placement Policies

Cache placement policies determine how memory addresses are mapped to cache blocks. Understanding these policies is crucial for optimizing cache performance. Below is a more detailed explanation of the three main cache placement policies: Direct Mapping, Set-Associative Mapping, and Fully-Associative Mapping.

Direct Mapping:- In direct-mapped caches, each memory address maps to a unique cache block. The address is divided into three components:

- **Offset:** Specifies the position within a cache block. This part is used to access the exact byte within the block.
- **Index:** Determines the cache line to which the memory address is mapped. The index is used to locate a specific line in the cache.
- **Tag:** Identifies whether the cache block contains the desired data. The tag is compared to the stored tag in the cache set to determine if a cache hit or miss occurs.

However, the main limitation of direct-mapping is that multiple addresses can map to the same cache line, causing evictions. For example, if two different memory addresses map to the same index, one will evict the other, resulting in a cache miss.

Set-Associative Mapping:- Set-associative mapping combines the simplicity of direct-mapping with the flexibility of allowing a cache set to contain multiple blocks (ways). In this case, a given memory address can be placed in any of the blocks within the set. In a 2-way set-associative cache, each set contains two blocks, allowing a memory address to map to either block within the set. This reduces the number of evictions compared to direct mapping because multiple blocks are available for each set.

The number of blocks per set (known as the “way”) can vary, with 2-way and 4-way being common examples. The increased flexibility helps improve cache hit rates and reduces the likelihood of conflicts that cause cache misses.

Fully-Associative Mapping:- In fully-associative mapping, there is only one cache set, but it contains all the cache blocks. This means any memory address can map to any block in the cache, maximizing cache utilization. There is no index field since there is only one set. This allows full utilization of the cache, meaning a block is only evicted when the cache is full. The main disadvantage of fully-associative mapping is that it requires searching through all the cache blocks to find a match, which can be computationally expensive. This increases both time and power consumption during cache lookups. The eviction policy in a fully-associative cache is typically based on criteria such as least-recently used (LRU), where the least recently accessed block is replaced when the cache is full.

E. Parts of a Memory Address

A cache consists of a *data array*, which stores data or code, and a *tag array*, which stores the high-order address bits. It is divided into *cache lines*, each of size B bytes. Typically, caches are set-associative, with S sets and W ways, where each cache line occupies one way of a set. The set is determined using address bits:

- The lower $\log_2 B$ bits represent the *block offset*.
- The next $\log_2 S$ bits form the *set index*.
- The remaining bits constitute the *tag*.

as illustrated in Fig 4

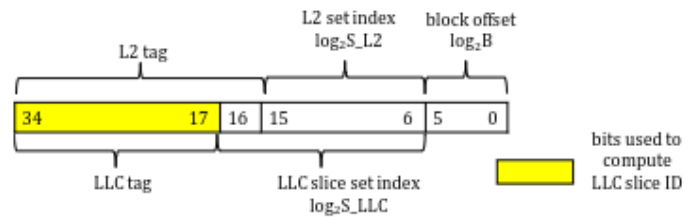


Fig. 4: Memory Address

When a core issues a load or store request, the tag array of the L1 cache is checked:

- On a *hit*, data is sent directly to the core.
- On a *miss*, the request moves to L2, LLC, and finally main memory.

Lower cache levels often have more sets than higher levels, leading to possible set mapping collisions in higher-level caches due to the pigeonhole principle. The *pigeonhole principle* states that if n items are placed into m containers, and $n > m$, at least one container must hold more than one item. In the context of caching, this means that when more cache lines map to a limited number of sets, multiple lines may occupy the same set, leading to potential conflicts.

The number of cache lines per set determined by the cache’s associativity. When a cache set is full and a new cache line must be loaded, the cache replacement policy, such as Least Recently Used (LRU) or First In, First Out (FIFO), evicts an existing cache line to make room for the new one.

F. Eviction Set

An **Eviction Set** is a collection of memory addresses that map to the same cache set. Accessing these addresses can cause all cache lines in the set to be evicted. To completely evict the cache lines, the Eviction Set must contain at least as many addresses as there are cache lines in the set. This ensures that all cache lines in the set will be replaced when the addresses in the Eviction Set are accessed. The size of the Eviction Set depends on the cache's associativity, as it must match the number of cache lines in the set to fully evict its contents.

G. Advanced Encryption Algorithm (AES)

1) **Introduction:** The AES algorithm is the state-of-the-art encryption algorithm which was developed by the National Institute of Standards and Technology (NIST) in 2001. It is a symmetric block cipher. It is called a block cipher because data or any set of bits are encrypted in the form of blocks of fixed size in order to produce ciphertext using a particular key and an algorithm. Also, it is symmetric because the same key is used for both encryption and decryption. It is one of the most widely used encryption techniques. There are 3 types of AES encryption schemes based on the size of the key:

- **128-bit AES** - The key size is 128 bits and it requires 10 rounds.
- **192-bit AES** - The key size is 192 bits and it requires 12 rounds.
- **256-bit AES** - The key size is 256 bits and it requires 14 rounds.

2) **AES Rounds:** A round is a set of operations that are performed on the bits to be encrypted or decrypted and each round has a different key which is obtained by performing transformations on the original key.

The structure of a 128-bit AES is shown in figure 5. A 128-bit or 16 byte plaintext (bits to be encrypted) is given to the AES hardware module and a 128-bit ciphertext or encrypted bits is obtained as the output. The 16 bytes are first stored in a 4x4 matrix or 2D array. The 4x4 matrix now undergoes an initial transformation. After that, it undergoes N rounds of transformation. The value of N varies based on the AES type as mentioned earlier and for 128-bit AES, the value of N is 10. The value of key used in this case is 128 bits or 16 bytes and it can also be arranged in the form of a 4x4 matrix. The key undergoes transformation inside the key expansion block and it produces the values of key for each round.

3) **AES Round Transformations:** Each round of the AES encryption and decryption consists of 4 transformations as follows:

- **Substitute Bytes** - This transformation involves substituting the location of each byte of data with another byte of data present elsewhere in the matrix. The mapping between which byte will be substituted by which other byte is defined using a 16x16 S-box as shown in figure 6.

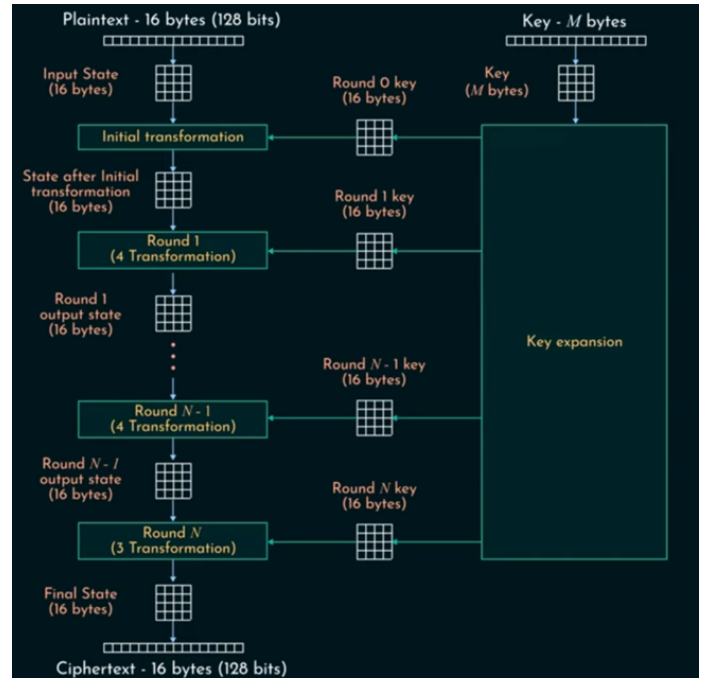


Fig. 5: AES Structure

- **Shift Rows** - In this transformation, the first row of bytes remains the same, the bytes in the second row are cyclically shifted by 1 column leftwards, the bytes in the third row are cyclically shifted by 2 columns leftwards and similarly the bytes in the fourth row are cyclically shifted by 3 columns as shown in figure 7.
- **Shift Columns** - In this transformation, the matrix obtained after the shift rows operations is multiplied by a 4x4 matrix which is defined by us and a new 4x4 matrix is obtained as shown in figure 8.
- **Add Round Key** - It is the last transformation and involves just adding or XORing the matrix obtained after the shift columns operation with the key for that particular round as shown in figure 9.

Note that first an add round key transformation takes place on the original plaintext matrix and then there are 10 rounds of transformations. Also, the first N-1 rounds consists of all 4 transformations but the last round consists of only 3 transformation and does not include the mix columns transformation as shown in figure 10. Moreover, the transformations involved during decryption are exactly in the opposite order as those involved during encryption.

4) **AES Key Expansion:** The AES Key expansion module consists of transformations performed on the original key in order to obtain the key required for each individual round. It consists of the same number of transformations as the number of rounds of AES. In each transformation, the 4 columns of the AES matrix are first given to a function g. This function includes various operations on the columns such as shifting the rows, substituting the columns, adding a row constant, etc

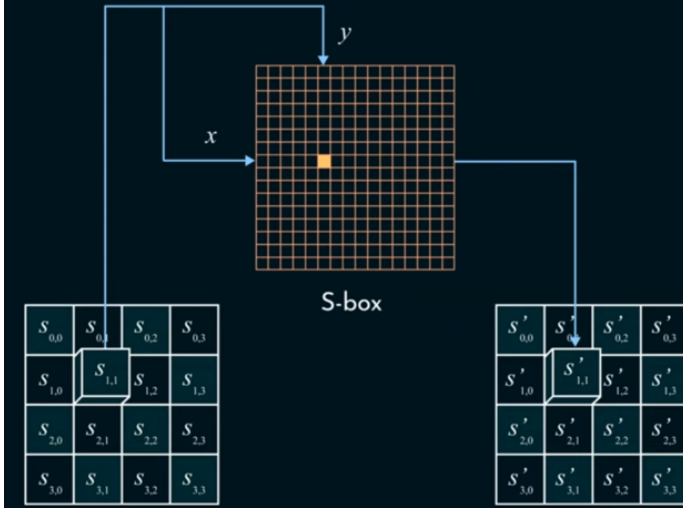


Fig. 6: Substitute Bytes Transformation

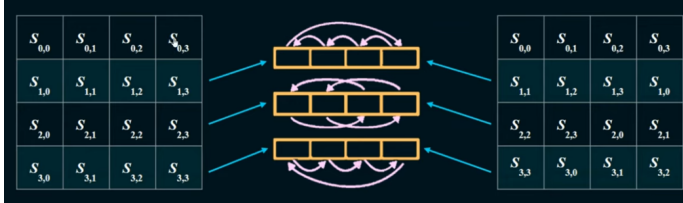


Fig. 7: Shift Rows Transformation

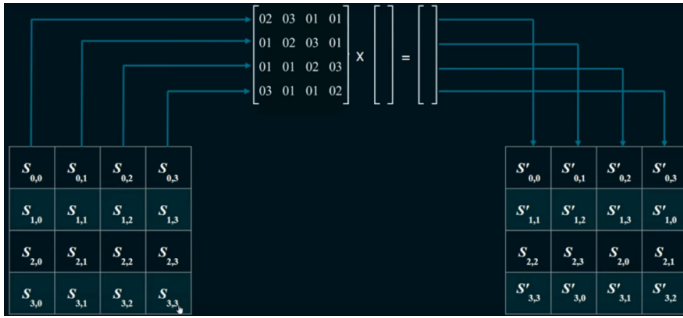


Fig. 8: Shift Columns Transformation

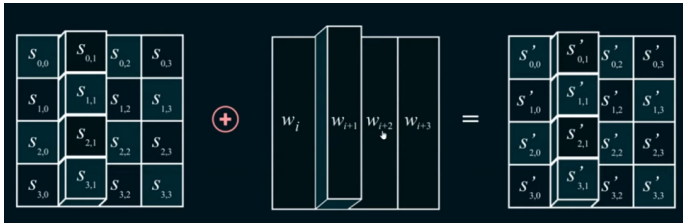


Fig. 9: Add Round Key Transformation

as shown in figure 11. The round constant is different for each round and is shown in figure 12. After this, the columns of the previous round and the transformed columns are added and then given to the next round of key transformation.

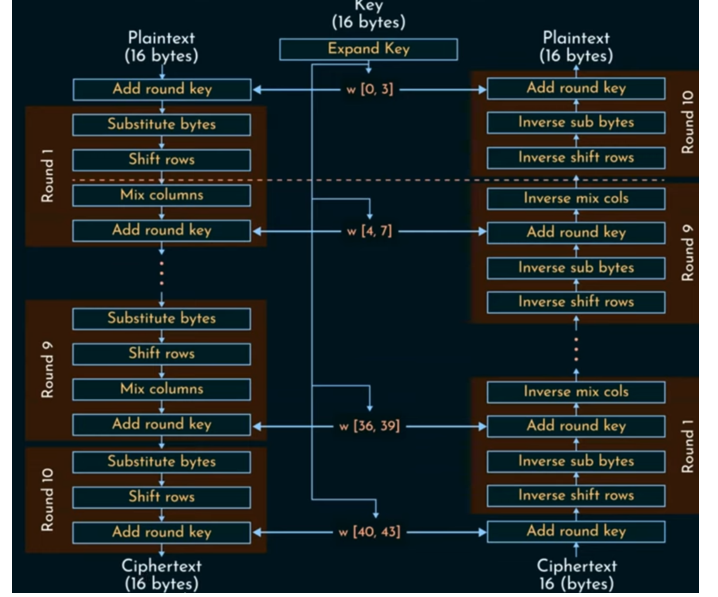


Fig. 10: Encryption and Decryption Rounds

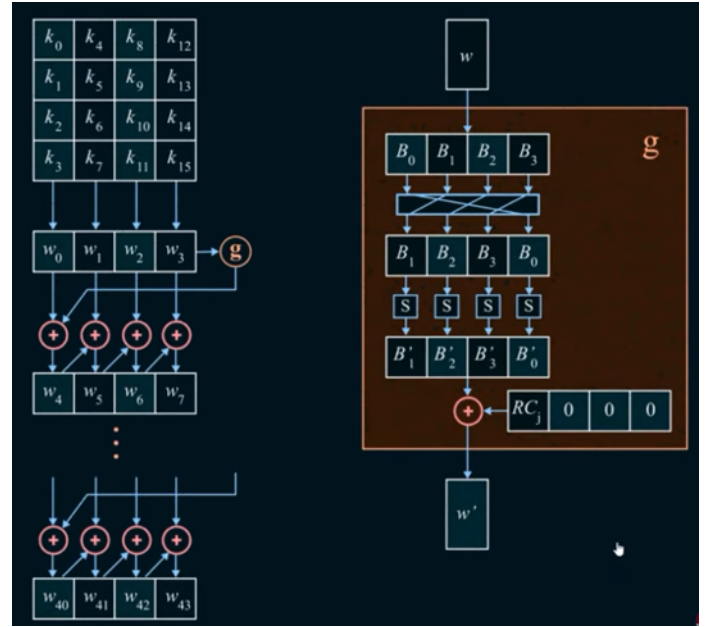


Fig. 11: AES Key Expansion Structure

II. CACHE-BASED SIDE CHANNEL ATTACK

A. Introduction

Cache-based side-channel attacks are a major security risk affecting devices like mobile phones, desktop computers, and servers. These attacks are hard to detect because they bypass regular software security measures. Attackers can gather both general user behavior and sensitive data such as cryptographic keys used in protocols like RSA and AES.

These attacks focus on the time it takes for the system to access specific memory locations. By analyzing these timing differences, attackers can figure out what the victim is doing.

Round	RC	Round	RC
1	(01 00 00 00) ₁₆	6	(20 00 00 00) ₁₆
2	(02 00 00 00) ₁₆	7	(40 00 00 00) ₁₆
3	(04 00 00 00) ₁₆	8	(80 00 00 00) ₁₆
4	(08 00 00 00) ₁₆	9	(18 00 00 00) ₁₆
5	(10 00 00 00) ₁₆	10	(36 00 00 00) ₁₆
Note: Initial Transformation takes (00 00 00 00) ₁₆ as the RC.			

Fig. 12: Round Constant(RC) values for AES Rounds

This makes such attacks especially dangerous when sensitive information like encryption keys or login details is being processed. Since these attacks depend on small timing variations, they are very hard to defend against and can compromise secure systems without direct interaction.

Algorithm 1 Square-and-multiply exponentiation

```

1: Input: base  $b$ , modulo  $m$ , exponent  $e = (e_{n-1} \dots e_0)_2$ 
2: Output:  $b^e \mod m$ 
3:  $r \leftarrow 1$ 
4: for  $i = n - 1$  downto 0 do
5:    $r \leftarrow \text{sqr}(r)$ 
6:    $r \leftarrow \text{mod}(r, m)$ 
7:   if  $e_i = 1$  then
8:      $r \leftarrow \text{mul}(r, b)$ 
9:      $r \leftarrow \text{mod}(r, m)$ 
10:  end if
11: end for
12: return  $r$ 

```

B. Example: Square-and-multiple exponentiation

An example of a cache-based side-channel attack is demonstrated using the square-and-multiply algorithm for exponentiation, commonly employed in encryption protocols like RSA and ElGamal, where the binary exponent e is targeted.

The attacker targets the memory addresses of the ‘sqr’ and ‘mul’ functions. The ‘sqr’ function is executed for each bit in the exponent e , whereas the ‘mul’ function is executed only if the corresponding bit e_i is 1.

To carry out the attack, the attacker evicts both the ‘sqr’ and ‘mul’ functions from the cache in every iteration of the loop by accessing other memory addresses that map to the same cache set or belong to the same eviction set, causing these cache lines to be removed. When the victim executes the exponentiation algorithm, timing variations occur. If $e_i = 1$, both ‘sqr’ and ‘mul’ will be executed, causing cache misses and slower access times. If $e_i = 0$, only ‘sqr’ is executed, resulting in faster access times.

By measuring these timing differences, the attacker can infer whether each bit in the exponent is 1 or 0, ultimately revealing the entire secret exponent e .

In the first step of a cache-based side-channel attack, the attacker identifies the target address, which can be achieved through source code analysis or techniques like cache template attacks. In the second step, the attacker gathers timing data to conduct the attack. This step follows three key actions:

- **Evict the Target Address:** The attacker forces the target address to be removed from the cache by accessing other addresses that map to the same cache set. This causes a cache miss when the victim accesses the target, leading to slower access times.
- **Wait for the Victim’s Access:** The attacker waits for the victim to access the target address during normal execution, without the victim’s knowledge.
- **Measure the Timing of Accesses:** The attacker either measures the time taken for the victim program to access memory or detect the time taken for attacker to access its data after victim program has run, which in turn will depend on the cache replacement based on the data accessed by the victim.

C. Flush+Reload Attack

The Flush+Reload attack is a cache-based side-channel attack illustrated in [7] that exploits the variations in memory access times to infer secret data. By measuring the latency of memory accesses, an attacker can determine whether a victim has accessed a particular memory address. This attack is possible in environments where the attacker and the victim share some memory space, such as variables or memory regions that are accessible to both parties. The attack relies on the attacker’s ability to manipulate the cache state and measure timing differences resulting from cache hits or misses.

The Flush+Reload attack proceeds in six steps as illustrated in Fig 13:

- 1) **Identification of Shared Memory:** The attacker identifies memory locations that both it and the victim shares using source code analysis or cache template attacks. (e.g., &a, &b).
- 2) **Flush Target Memory:** The attacker uses the `clflush` instruction offered by some CPUs to flush the target memory (e.g., &a) from the cache, ensuring that any subsequent access will result in a cache miss and fetching data from DRAM.
- 3) **Wait for Victim’s Execution:** The attacker waits for the victim to execute, during which the victim may access shared memory. If the secret is ‘1’, the victim accesses &a; if ‘0’, they access &b.
- 4) **Victim Accesses Shared Memory:** The victim accesses one of the shared memory addresses, causing a cache miss and fetching the data from DRAM.
- 5) **Reload Target Memory:** After the victim executes, the attacker accesses the same memory addresses and measures access latency.
- 6) **Timing Analysis:** The attacker compares access timings. A fast access (cache hit) suggests the victim accessed the memory and the secret is ‘1’; a slow access (cache miss)

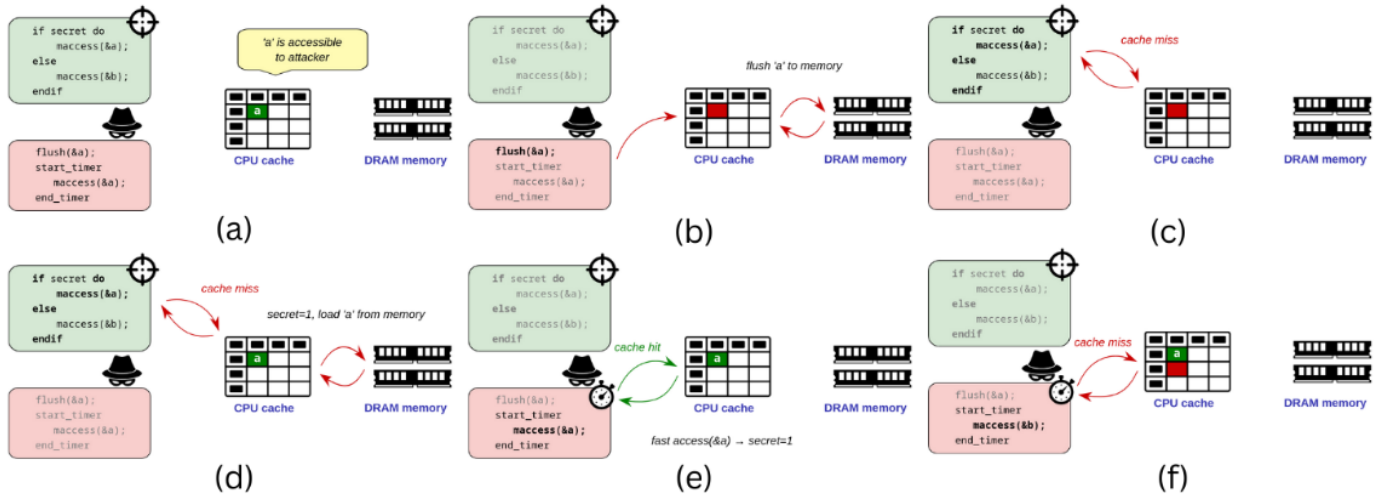


Fig. 13: Flush+Reload Attack

suggests the victim did not access the memory, meaning the secret is '0'. Multiple rounds of this procedure help infer the secret value.

However, it requires specific hardware support for cache flushing (not available on all CPUs) and assumes a predictable memory access pattern from the victim's program. The attack is effective when shared memory access is possible, such as in certain cloud environments with limited cache isolation.

Flush+Reload is a general cache-side channel attack. Cache Address Encryption is a solution for a category of cache-side channel attack called conflict-based cache-side channel attack. For doing these attacks, we need to define what a minimal eviction set is.

D. Minimal Eviction Set

To perform a conflict-based cache side-channel attack, attackers first need to create an address conflict in the cache. This is done by identifying a "minimal eviction set," which is a small group of cache lines that can reliably cause evictions. Attackers start with a larger candidate set of cache lines and narrow it down through testing until they find the minimal eviction set.

To find this set, attackers load data into the cache and then trigger evictions by accessing other addresses. By observing which cache line gets evicted, they gather important information for the attack.

The purpose of the minimal eviction set is to create conflicts in the cache. When a cache miss happens, the cache replaces an old line with new data. If two addresses map to the same cache set, this replacement causes a conflict. Attackers use this behavior to gather information about the victim's memory access patterns.

E. Evict+Reload Attack

The Evict+Reload attack described in [7] is similar to Flush+Reload, but it does not use the `clflush` instruction for eviction. Instead, it relies on cache conflicts to evict the target

address. The attacker accesses a minimum eviction set having address it has access to, mapped to the same set as the target address causing the target to be evicted. Afterward, the attacker monitors the cache for reloads, just like in Flush+Reload. The key difference is that Evict+Reload does not require `clflush`, making it suitable for systems where `clflush` is unavailable or restricted. Despite the added complexity of inducing cache conflicts, this attack works similarly to Flush+Reload, inferring sensitive information through cache behavior.

Both Flush+Reload and Evict+Reload attacks rely on shared memory between the attacker and the victim. If data sharing between processes is disabled, as implemented in many modern cloud platforms, these attacks become ineffective.

F. Prime+Probe Attack

The Prime+Probe attack is a cache side-channel attack that allows an attacker to infer information about the victim's memory accesses, without requiring shared memory, unlike previously discussed attacks. This attack works by exploiting cache evictions and timing differences.

In a Prime+Probe attack, the attacker and victim execute on the same machine, without shared memory. The victim has access to a variable a , and the attacker has access to a variable c that maps to the same cache line as a . Figure 14 illustrates the different stages in the attack. Here's how the attack unfolds:

- 1) **Prime Step:** The attacker evicts the address a from the cache by accessing the address c . This primes the cache by causing the eviction of the cache line containing a , replacing it with data from c .
- 2) **Wait Step:** The attacker allows the victim to execute its operations. Assuming the victim accesses a and `secret == 1`, the victim's request for a causes a cache miss because it was evicted in the prime step. The system fetches the data for a from DRAM.
- 3) **Cache Miss and Eviction:** The data for a is placed in the cache after fetching it from DRAM. This causes the

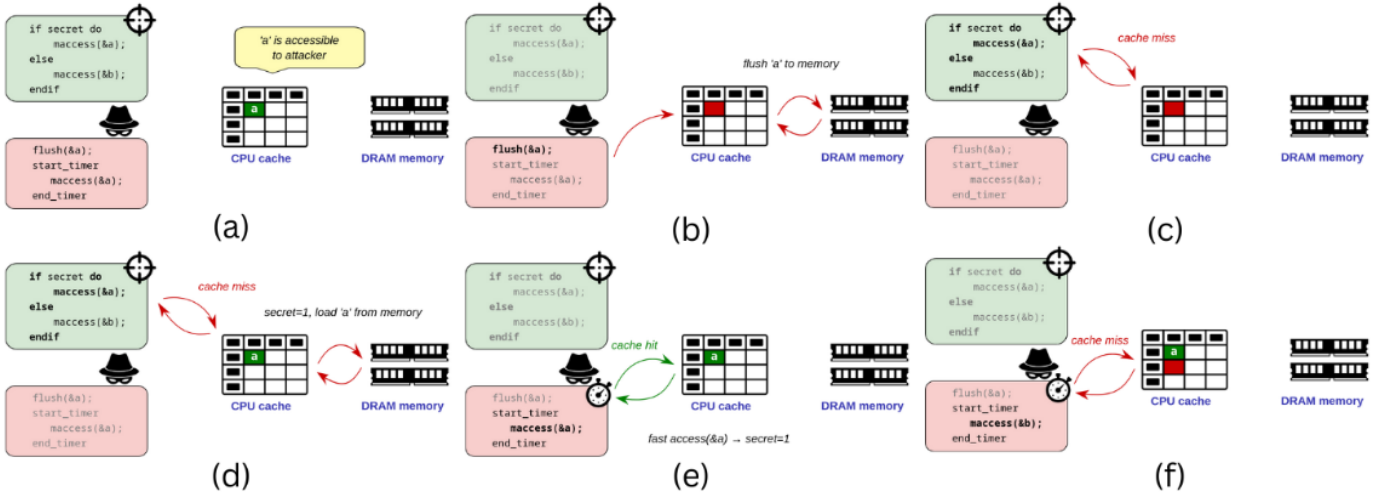


Fig. 14: Prime+Probe Attack

eviction of the attacker's data c from the cache, which is now replaced by a .

- 4) **Probe Step:** After the victim executes, the attacker probes the cache by accessing c again and measuring the access time. If the access is slow (cache miss), it indicates that the attacker's data was evicted from the cache when the victim accessed a .
- 5) **Inference:** If accessing c is slow, it indicates that a was accessed by the victim, meaning the secret is 1. Otherwise, the secret is 0.

A broader explanation of the Prime+Probe attack is provided below. The attacker selects a set of addresses, known as the minimal eviction set, that map to the same cache set as the victim's target address a . By filling the cache with these addresses, the attacker ensures that the victim's target address a is evicted from the cache. If the secret value is '1', the victim accesses a , causing a cache miss and evicting one of the attacker's addresses from the minimal eviction set. If the secret is '0', the victim does not access a , and no eviction occurs.

After the victim executes, the attacker probes the cache by accessing the addresses in the minimal eviction set. If the secret is '1', one of the attacker's addresses has been evicted, causing a cache miss and a slower memory access time when the attacker probes it. On the other hand, if the secret is '0', no eviction has occurred, and the attacker's access will be faster, as no cache miss has happened. By timing these accesses, the attacker can infer whether the target address a was accessed, thus revealing the secret information.

The **Prime+Probe** attack is a powerful method for observing and inferring the memory access patterns of a victim, without needing shared memory between the attacker and the victim. By carefully timing cache accesses, the attacker can determine which addresses the victim is accessing, potentially exposing sensitive information. This technique exploits cache conflicts and evictions, making it a potent tool for conducting side-channel attacks in systems where shared memory is not

required.

The **Evict+Reload** and **Prime+Probe** attacks are called conflict-based cache side channel attacks as they exploit conflicts in cache structures to get access to sensitive data. These attacks exploit cache evictions to infer the victim's memory access patterns, which could lead to extracting sensitive information such as cryptographic keys or other secrets. The attacker relies on the difference in timing between cache hits and misses to determine if the victim has accessed the target address during the execution of the victim's code.

III. SOLUTIONS AND CONSTRAINTS

A. Address Encryption

Encrypting the mapping from physical addresses to cache addresses is a common technique to defend against conflict-based cache side-channel attacks as proposed in [3][2]. By obscuring the physical-to-cache address mapping, the random or encrypted mapping ensures that data occupying the same cache line will have different indices. This prevents attackers from evicting specific cache lines, making it difficult for them to identify minimum eviction set.

Fig 15 illustrates the structure of address encryption mechanism proposed in [6]. The address encryption mechanism encrypts all parts of the physical address except the offset. The solution is illustrated using the XOR algorithm for encryption. In this mechanism, each cache way, is equipped with an encryption module such as Encryption0 through EncryptionM,t that applies a unique encryption process to the requested address. When a data request is made for a physical address x , the address is encrypted using a random cipher to generate a new encrypted index, $EIndex$. This encrypted index determines the location of the data in the cache, ensuring that the cache address mapping is obfuscated. The encryption process effectively eliminates any predictable patterns in the address mapping, making it difficult for attackers to construct minimum eviction sets, which are sets of addresses that map to the same cache index.

The process begins by dividing the requested address into multiple cells, denoted $E_1, E_2, \dots, E_T$, where each cell corresponds to a portion of the address that aligns with the cache's index bits. A random number generator produces a series of random ciphers $E_{r_1}, E_{r_2}, \dots, E_{r_a}$, where a is the associativity of the cache. These ciphers are used as keys for encrypting the address cells. Each address cell is XORed with the ciphers E_{r_i} , generating intermediate values. The sum of these intermediate values produces the final encrypted index, E_{Index_i} , which is used to determine the cache location. Even a minor change in the address results in a significantly different E_{Index} , adding complexity to any attack attempt.

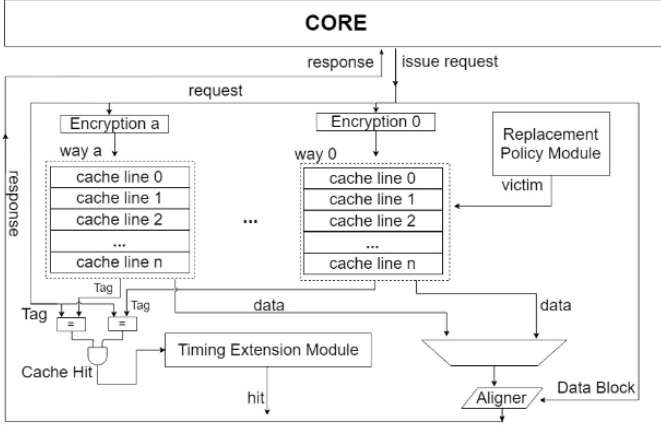


Fig. 15: Structure of Address Encryption

To further enhance security, the encryption module periodically updates a subset of the random ciphers using a randomization function, ensuring that the mapping is dynamic and the attacker cannot predict or exploit the mapping over time. Only a subset of ciphers is updated at each interval to minimize performance overhead, maintaining system efficiency.

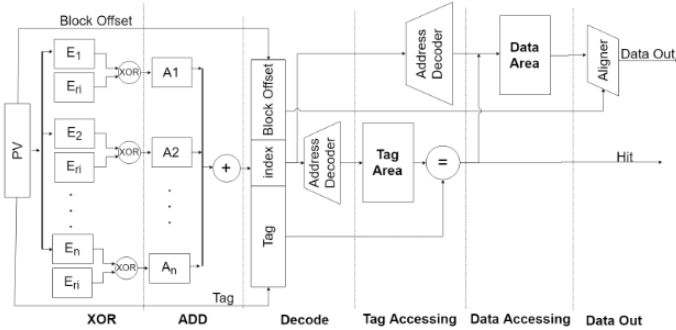


Fig. 16: Serialised access of address encryption mechanism

This encryption technique is seamlessly integrated into a set-associative cache. Each cache way has its own dedicated encryption module, which uses unique random ciphers to secure the cache index mapping. The encryption process introduces a minimal delay, typically two clock cycles, consisting of one clock cycle for the XOR operation and another for

summing the results. Once the encrypted index is computed, it is used to access both the cache's tag and data areas, ensuring that the cache functions as a standard set-associative cache while offering enhanced protection against conflict-based cache side-channel attacks.

The primary advantage of this encryption mechanism is its resistance to cache-based side-channel attacks. By encrypting the mapping from physical addresses to cache indices, attackers are unable to predict which cache lines correspond to specific addresses, making the eviction set highly complex and difficult to exploit. Furthermore, periodic updates to the random ciphers add an additional layer of security, ensuring the mapping remains unpredictable and resistant to attack over time.

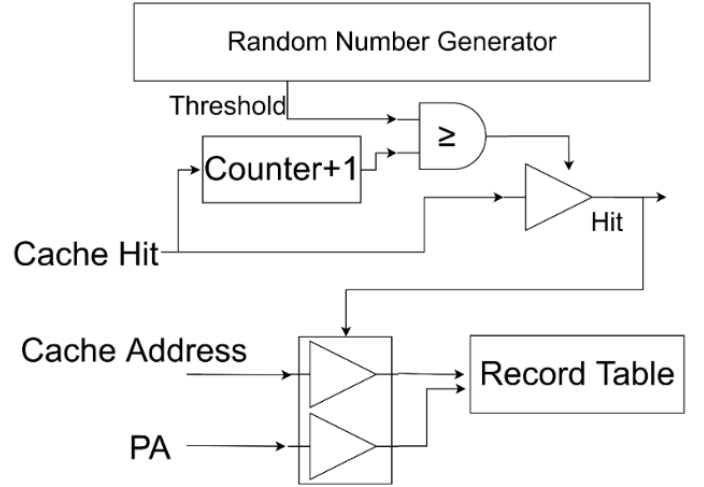


Fig. 17: Timing Extension Mechanism

B. Time Extension Mechanism

Encrypting the mapping between physical addresses and cache addresses can make it harder for attackers to identify minimal eviction sets but does not completely prevent them from pruning the set. Some attacks do not rely on a minimal eviction set, or may not require a high eviction rate (e.g., 99%). So, attackers can still succeed.

So, a *timing noise extension mechanism* as illustrated in Fig 17 and proposed in [6] is introduced. This mechanism includes an independent circuit that monitors cache requests. Once the number of accesses exceeds a certain threshold (determined by a random number generator), the mechanism triggers a cache miss and forwards the request to the next memory level. A counter tracks cache hits, and when the threshold is reached, it forces a cache miss, resetting the counter. The random noise can cause data reloading, which is resource-intensive, but a *record table* helps mitigate this by tracking cache lines and their physical addresses, allowing the cache to reuse lines efficiently when data is reloaded.

IV. IMPLEMENTATION APPROACH

The Advanced Encryption Standard(AES) algorithm can be implemented on a RISC-V Processor to improve the security and confidentiality of data stored in the processor. For the purpose of this project, the Shakti RISC-V processor can be used. It is an open-source initiative by the Reconfigurable Intelligent Systems Engineering (RISE) group at the Indian Institute of Technology, Madras (IIT Madras). The Shakti processor has several variants available such as E-class, C-class, I-class, M-class, H-class and S-class. However, the later 4 variants are still in development. For this project, the C-class Shakti processor can be used. It is a controller class of processor which is aimed at mid-range application workloads. The core is highly optimized, 5-stage in-order design with MMU support and capability to run operating systems Linux and Sel4. It targets compute/control applications in the 0.5-1.5 Ghz range.

A. Shakti RISC-V Processor Core

The micro-architecture of C-class processor is shown in figure 18. It has 5 stages namely Fetch, Decode, Execute, Memory and Writeback. The complete code for this processor is given in the GitLab repository[4] of the Shakti processor. The Shakti processor has been coded using the Bluespec SystemVerilog language. Within the GitLab repository, if we first go to cores followed by c-class and then src, we can find all the codes for the 5 stages of the processor.

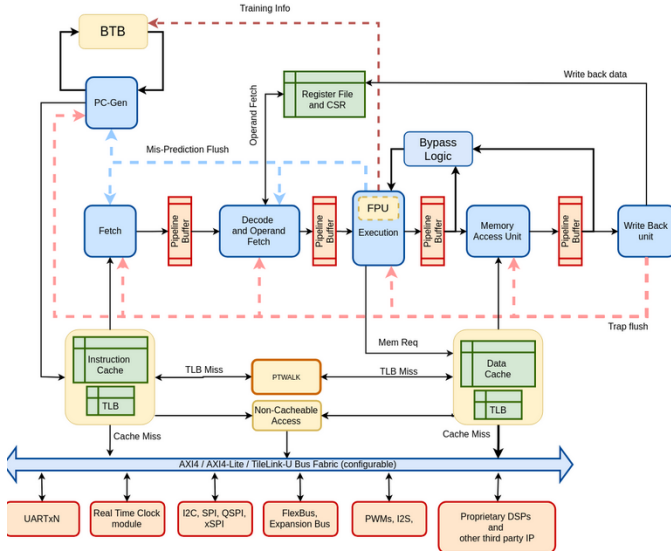


Fig. 18: C-class Shakti Processor Micro-Architecture

Timeline - 3-4 days to understand the different packages present in the C-class core and their features and working in detail.

B. Shakti RISC-V Processor Cache

The C-class Shakti processor has separate data and instruction caches and their corresponding TLBs. The internal structure of the L1 cache is shown in figure 19.

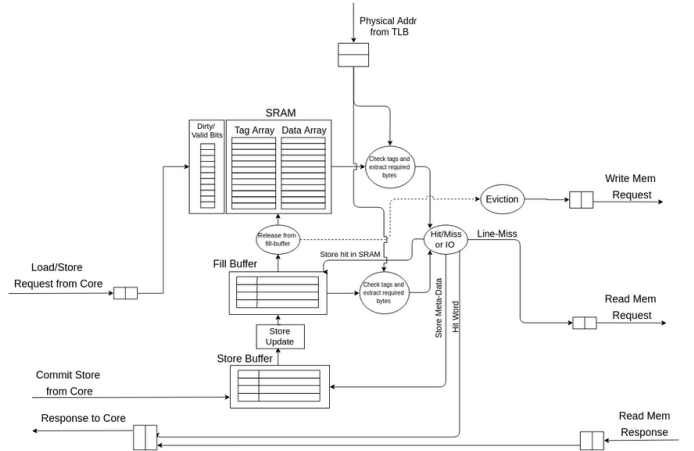


Fig. 19: C-class Shakti Processor L1 Cache

The codes for the caches can be found in the GitLab repository by first going to 'uncore' followed by 'caches_mmu' and then 'src'. It contains all the codes for data cache, instruction cache and the last level cache(LLC).

Timeline - 2-3 days to understand the working of all the packages present in the cache.

C. AES Package

The AES algorithm has already been implemented in BlueSpec SystemVerilog and is openly available in the Accel_AES GitHub repository[1]. It is a hardware IP designed in BSV, but this repository also contains pre-generated Verilog that can be used immediately. The IP consists of 3 layers in the following top-to-bottom hierarchy:

- **Layer 0:** AXI4 master and slave adapter - It adapts the generic memory slave and master interfaces in Layer 1 into a 64-bit AXI4 slave and master, respectively. By replacing Layer0, one can use the IP with other types of buses. However, in this project, we won't require this layer.
- **Layer 1:** Generic Memory-to-Memory AES accelerator - This is a wrapper for the core AES hardware, converting it into an IP block for an SoC so it can behave as a memory-to-memory AES accelerator. It uses Layer 2 to implement a memory-to-memory functionality with memory master and slave interfaces that are completely generic (no specific bus protocol). In this project, this layer is not required as the C-class is a simple chip processor. However, the AES algorithm can be easily extended to RISC-V SoCs using this layer.
- **Layer 2:** AES core - It is the inner-most layer (AES core) of the implementation of AES. Conceptually, it is an object with 3 methods:
 - 1) Set key (128-bit)
 - 2) Encryption block (128-bit)

3) Decryption block (128-bit)

The AES layer 2 consists of the following packages:

- **AES_Defs** - It defines the common types and values for the BSV code for implementing AES hardware.
- **AES_Encrypt_Decrypt** - It implements the core functions of the AES algorithm:
 - 1) Key expansion of a 128-bit key
 - 2) Encryption of a 128-bit block using that key
 - 3) Decryption of a 128-bit block using that key
- The AES 'rounds' are done sequentially, with each round transforming a single 'state' register.
- **AES_IFCs** - It is used to supply a key on the 'set_key' interface and to encrypt/decrypt one or more blocks of data.
- **AES_KeyExpand** - It is used for the key expansion used for AES algorithm.

Timeline - 1-2 days to understand the working of the AES package.

D. Integrating AES in Shakti Processor

As described in part (IV) B, the 'src' folder within the 'un-core' folder in the Shakti processor GitLab repository contains the BSV codes for all the parts of data and instruction cache. Within the 'dcache' folder, we can find the 'dcache_types.bsv' file. The package present in the file defines various types, functions and structures related to a data cache system. It is structured to handle different aspects of cache and memory operations, including access requests, memory responses, and atomic operations. It contains the 'DMem_request' typedef as shown in the figure 20. It defines the structure of a memory request sent to the data memory subsystem. Some of its fields are as follows:

- **address:** It gives the memory address to be accessed for load, store or atomic operation.
- **epochs:** It is used for coherence or consistency in memory transactions.
- **size:** It specifies the size of the data being accessed.
- **access:** It gives the type of memory access, that is, Load, Store or Atomic).

Also, as described in part (IV) A, the 'src' folder present within the 'cores' folder in the Shakti processor GitLab repository contains the BSV codes for all the packages for the various stages of the processor. Within this folder, we can find the 'pipe_ifcs.bsv' file. The package defined within it contains the inter-connections of the different stages of the processor as well as with the cache and other peripherals. The 'Ifc_s3_cache' interface present within it acts as an interface to send memory requests to the 'DMem_request' interface within the dmem subsystem as shown in figure 21. Therefore, while loading data from cache, the cache address is given to 'DMem_request' along with the appropriate access mode and it returns the data stored. Also, in order

```
typedef struct{
  Bit#(addr)    address;
  Bit#(esize)   epochs;
  Bit#(3)       size;
  Bool         fence;
  Bit#(2)       access;
  Bit#(data)    writedata;
  Bit#(2)       prv;
`ifdef atomic
  Bit#(5)       atomic_op;
`endif
`ifdef hypervisor
  Bool         hfence;
  Bit#(1)      virt;
  Bit#(1)      hlvx;
`endif
`ifdef supervisor
  Bool         sfence;
  Bool         ptwalk_req;
  Bool         ptwalk_trap;
`endif
} DMem_request#(numeric type addr,
               numeric type data,
               numeric type esize ) deriving(Bits, Eq, FShow);
```

Fig. 20: DMem_request typedef in the dcache_types.bsv file

to store a data in the cache, the data and address is given to 'DMem_request' along with its corresponding access mode.

```
interface Ifc_s3_cache;
// interface to send memory requests to the dmem subsystem
interface Get#(DMem_request#(`vaddr, `elen, 1)) mv_memory_request;
(*always_enabled*)
method Action ma_cache_is_available(Bool avail);
endinterface:Ifc_s3_cache
```

Fig. 21: Ifc_s3_cache and DMem_request interfaces in pipe_ifcs.bsv file

Now, the AES packages given in Accel_AES repository[1] can be inserted between the memory stage interface of the processor and the data cache interface 'DMem_request' such that during store, the address location of the data to be stored is given to the AES package, then the AES package encrypts this address and stores the data in the corresponding encrypted address. Similarly, in order to load a data from cache, the memory interface can give the encrypted address to the AES package, the AES package can then decrypt this address and then send a request to the 'DMem_request' interface to get the data from that address location.

Timeline - 6-7 days as it may require some time to understand the integration of the core package, AES package and the cache package and to make certain changes to these packages if required.

E. Implementation of the Modified Processor using FPGA

After integrating the AES algorithm and timing extension mechanism in the Shakti RISC-V processor, the processor can now be implemented on an FPGA(Field Programmable

Gate Array) in order to check the overall working as well as the working of the AES algorithm. Also, we will need to integrate the FPGA with Linux operating system to run the code on it.

Timeline - 3-4 days to learn about FPGA and then to run the processor on it.

F. Testing the Modified Processor

After implementing the modified processor on an FPGA, a code can be run on it to simulate a Prime and Probe attack, which is a type of cache timing attack. The code for the Prime and Probe attack is given in the 'nagnagnet' GitHub repository[5]. It contains a 'main.c' code which is the attacker code and a 'key.c' code which is the victim code. The attack is executed in the following manner:

- **Step 1 - Prime Phase** - As shown in the code written in Figure 22, the conflict set is created using the loop. Also, the probe function is called repeatedly to detect access times and determine whether an address evicts a target cache line or not. If it causes a cache miss, it is added to the conflict set. Note that the cache hit and cache miss are determined by comparing the access time with certain threshold values.
- **Step 2 - Execution of Victim program** - The victim program performs repeated access to buf[0].num as shown in figure 24 and iterates over the entire array buf in specific patterns. This evicts some of the attacker's cache lines from the primed cache set. As a result, the victim loads certain data in those cache lines by utilising the cache replacement policy.
- **Step 3 - Probe Phase** - As shown in the code present in figure 23, the probe phase is implemented by the loop and RDTSC(Read Time Stamp Counter) is used to measure the access time. The attacker uses the probe() function to find the access time for each line in the primed set.
- **Step 4 - Inference based on Access Time** - The access time results are written to a file (data.txt) for analysis. Repeated cache misses on specific cache lines suggest that the victim accessed those lines. Hence, based on that the secret data can be found out.

Now, this code can be run on the FPGA and we can check whether we are able to get the data or not and whether that data matches the secret data that has been stored in the memory.

Timeline - 1-2 days in order to understanding the detailed working of the code and testing for various cases.

V. POTENTIAL CHALLENGES IN IMPLEMENTATION

- Some parameters of the Shakti processor and the AES algorithm might be different. Hence, it might be required to make few changes to both the processor and the AES implementation codes.

```
for(j = 0; j < 20; j++){
    struct st *conflict;

    conflict = (struct st*)malloc(sizeof(struct st));
    conflict->num = CONF;
    conflict->next = conflict->prev = conflict;

    shuffle(lines, j, N);

    for(i = 0; i < N; i++){
        n = lines[i];
        //printf("%d ", n);
        if(!probe(conflict, &buf[n*SET + set], 1)){
            buf[n*SET + set].count++;
            insert(conflict, &buf[n*SET + set]);
        }
    }
    free(conflict);
}
```

Fig. 22: Prime Phase code

```
for(i = 0; i < 8; i++){
    for(j = 0; j < SET; j = j + 64){
        for(l = 0; l < (L/8); l++){
            reader = buf[j + evi_buf[(L/8)*i+1].head].num;
        }
        for(k = 0; k < ATTACK; k++){
            loop();
            asm("lfence");
            RDTSC(t1);
            for(l = 0; l < (L/8); l++){
                //Prime & Probe phase
                reader = buf[j + evi_buf[(L/8)*i+1].head].num;
            }
            asm("lfence");
            RDTSC(t2);
            time = t2 - t1;
            array[k] = time;
        }
        for(k = 0; k < ATTACK; k++){
            fprintf(fp, "%d ", array[k]);
        }
        fprintf(fp, "\n");
    }
}
```

Fig. 23: Probe phase code

- The AES implementation given in the Accel_AES repository is used for 128-bit encryption. However, if we want to use 192 or 256-bit encryption techniques then few changes would be required in the AES implementation including increasing the number of rounds.
- Implementing a single cycle AES might be easy however implementing a multi-cycle AES might be challenging and would require adding some additional hardware.
- In order to implement the modified processor on an FPGA or a hardware simulator, we might require the integration

```

while(1){
    for(i = 0; i < N; i++){
        for(j = 0; j < 25000*10; j++) n = buf[0].num;
        if(i % 2 == 0){
            for(j = 0; j < N; j++){
                for(k = 1; k < 20*25000/N; k++) n = buf[j].num;
            }
        } else {
            for(j = 0; j < N; j++){
                for(k = 1; k < 20*25000/N; k++) n = buf[j].num;
            }
        }
    }
}
return 0;

```

Fig. 24: Victim Execution Code

of Linux operating system with the processor to run the code which will be used to test the processor and AES.

- The testing methodology which we have proposed may or may not work due to the simulator or processor constraints.

VI. OBSTACLES FACED

- Clear and complete understanding of the problem statement.
- Understanding the working and significance of the different types of cache timing attacks in detail.
- Understanding the different packages and interfaces present in the Shakti processor and the AES algorithm implementation and finding methods to integrate AES with the Shakti processor was challenging.
- Finding a method to test the working of the AES in preventing cache timing attacks once the implementation is done.

VII. LIMITATIONS AND REMAINING CHALLENGES

- Cache Address Encryption cannot prevent the Flush + Reload attack. Hence, there is a need to find an alternative solution for it.
- Currently, AES algorithm is meant for only 128, 192 and 256 bits but the cache address will be 64 bits and we encrypt only the index bits of the address. If we want to use 128-bit AES, we can zero-pad the address to make it 128 bit. But in order to avoid this, there is a need for the development of AES for lesser number of bits such as 64 or 32.
- The AES algorithm is complex and resource-intensive, requiring substantial hardware, power, and time for execution. Its high latency can significantly affect processor performance, particularly in programs with heavy memory access demands. Therefore, reducing the latency of the AES algorithm or exploring alternative encryption methods with lower latency is essential. Lightweight and low-latency encryption algorithms, such as Prince and Feistel, may be more suitable for this application.

VIII. CONTRIBUTION

- Currently, the Shakti processor relies on data encryption methods. However, it might not be sufficient and hence the cache address encryption technique can significantly increase the level of security of the processor.
- The Shakti processor uses lightweight encryption techniques such as Prince and Fiestel. However, those techniques might be sufficient only for IoT or such basic applications. In order to build an SoC, a much better encryption algorithm such as AES would be required.

IX. ACKNOWLEDGEMENT

We would like to thank our professor Dr. Suresh Purini for giving us a chance to work on such an interesting and practical project. We would also like to thank him for providing guidance whenever required and giving us helpful and valuable feedback at each stage of the project.

REFERENCES

- [1] Bluespec. Accel_aes. https://github.com/bluespec/Accel_AES, 2019.
- [2] Fangfei Liu and Ruby B. Lee. Random fill cache architecture. In *Proceedings of the 47th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO-47*, page 203–215, USA, 2014. IEEE Computer Society.
- [3] Fangfei Liu, Hao Wu, Kenneth Mai, and Ruby Lee. Newcache: Secure cache architecture thwarting cache side-channel attacks. *IEEE Micro*, 36:8–16, 09 2016.
- [4] IIT Madras. shakti. <https://gitlab.com/shaktiproject>, 2024.
- [5] nagnagnet. Primeprobe. <https://github.com/nagnagnet/PrimeProbe>, 2018.
- [6] Dehua Wu, Sha Tao, and Wanlin Gao. Applying address encryption and timing noise to enhance the security of caches. *Electronics*, 12(8), 2023.
- [7] Mengjia Yan, Read Sprabery, Bhargava Gopireddy, Christopher Fletcher, Roy Campbell, and Josep Torrellas. Attack directories, not caches: Side channel attacks in a non-inclusive world. pages 888–904, 05 2019.